Figure 1: Choose Source Database

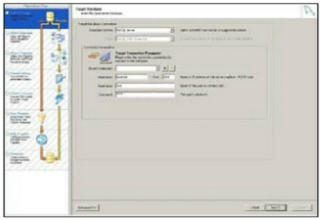

Figure 2: Choose Target Database

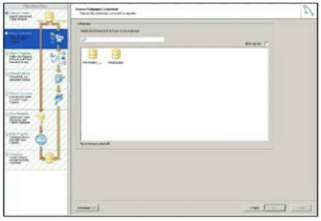

Figure 3: Choose Schema to migrate

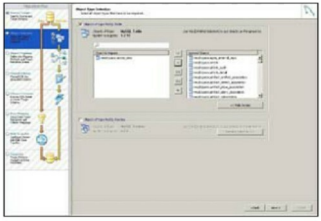

Figure 4: Choose object to migrate

MySQL is an open source solution. Licensing costs, if any, aren't high.

- **Estimated life of hardware and software:** The life of hardware is limited; so is the software's licence period. Consider your existing infrastructure and then plan for the future environment before deciding on a database solution.
- **Integrated new systems:** Consider MySQL if you want integrated new systems with low investments.
- **Feature support:** RDBMS technologies are ever expanding and have a horde of features. MySQL supports all the major features—replication, partitions, clusters, etc.
- **Community and industry trends:** The reach and acceptance of MySQL is high due to the above reasons. You are not making a risky decision when choosing MySQL. Though, don't follow the herd.

Other than these reasons, there can be many factors—availability of support, in-house skill sets, etc, but these can be built up if they do not exist. In my experience, it is the architecture of your project and your performance requirements that will define the choice of your solution. Under these criteria, in my experience, MySQL makes a fine choice for most migration problems.

Next up: how do we migrate?

Migration checklist

Yes, migration can be a very tricky job. You have to manage all data types, create relations, map data, and process huge chunks of mission-critical data. The margin of tolerance for any mistakes and errors is zilch. I use a checklist that guides me through the potentially troubled times. You should have one handy too. But before that, plan for your migration efforts. Ask yourself the following questions:

1. What are the characteristics of my application? Is it an OLTP (online transaction processing) application, a data warehouse, etc?
2. How quickly do I need to migrate? What are the timelines? Is it weeks or months?
3. Are there any tools that are available to help me perform the migration? ETL (extract, transform and load), scripts, ER (entity-relationship) tools, documentation?
4. How many objects will be migrated? Tables, sequences, indexes, etc?
5. How large is the database? Is the size in megabytes, gigabytes or terabytes?
6. Am I looking forward to increased capacity or scalability? By how much?
7. How many concurrent users will be present?